

Socius Workflow Logic

Editable product workflow specification for the Socius moderated community platform.

Socius is a moderated community platform. Users browse public communities and posts, members create posts, moderators approve or reject content, and admins manage the platform.

1. Public Visitor Flow

A visitor starts at the root URL and is redirected to the discovery page.

```
/ -> /discover
```

Public users can:

- View /discover with platform stats, featured/recent approved posts, and featured communities.
- Search communities from the top bar.
- Open /community to see all communities.
- Open /community?slug=community-name to view one community, its rules, member count, post count, and approved posts.
- Open /post?id=123 to read a single approved post and its comments/replies.
- Visit /about, /contact, /login, and /register.

Public users cannot create posts or join communities until they log in.

2. Authentication Flow

Users register at /register. Registration collects username, email, phone number, password, and confirm password.

New user defaults:

- role = member
- display name = username
- bio = empty

```
Successful registration: /register -> /login
```

Login accepts username or email at /login.

Role redirects after login:

- member -> /member/home
- moderator -> /moderator/dashboard
- admin -> /admin/dashboard

Protected route rules:

- /member/* requires login
- /moderator/* requires moderator or admin
- /admin/* requires admin

3. Member Home Feed

Member home is /member/home. This is the main feed page and should work like a calm Reddit-style front page.

Current feed logic:

- Load all approved posts from all communities.
- Order by featured first, then newest.
- Show each post as a feed card.
- Allow the user to scroll through all approved posts.

Each feed post shows:

- community avatar or initial
- post title
- community name
- author username
- optional image
- content preview
- upvote count
- comment count
- link to full discussion

```
Click post -> /post?id=postId  
Click community -> /community?slug=communitySlug
```

The right side of the feed shows a profile dashboard shortcut and suggested/trending communities.

4. Member Profile Dashboard

The old dashboard logic conceptually lives in /member/profile. This page combines personal account information and activity health.

Profile dashboard includes:

- total user posts
- pending review posts
- joined communities
- penalty points
- profile details form
- recent publishing queue
- account security notes

Current limitation: the profile form is visual only right now and does not yet save profile edits to the database.

5. Member Content Flow

Members create posts at /member/create-post.

Create post form includes:

- community selector
- post type: text/resource/event/image
- title
- content
- optional image upload
- image alt text
- Save draft button
- Submit for review button

Post submission logic:

- Save draft -> status = draft
- Submit for review -> status = pending

Validation:

- community is required
- title must be clear and at least 5 characters
- post must include text or an image
- image must be JPG, PNG, GIF, or WebP
- image max size is 5 MB
- banned users cannot post in that community

After saving/submitting: /member/create-post -> /member/my-posts

6. Member Community Flow

Members create communities at /member/create-community.

Community form includes:

- name
- description
- guidelines

Create community logic:

- validate name and description
- generate unique slug from name
- create community
- requires_review = true
- creator automatically joins the community
- redirect to /community?slug=new-community

Join community flow:

```
/community?slug=example  
-> Join community  
-> POST /member/join-community  
-> membership is created  
-> redirect back to community page
```

7. Search Flow

The top bar search supports partial live suggestions.

```
Typing into search calls: /search-suggestions?q=query
```

Suggestions return JSON for:

- matching communities
- matching approved posts

Community search checks:

- community name
- community description

Post search checks:

- post title
- post content
- community name

Clicking a suggestion navigates directly to that community or post. Submitting the search form still goes to `/community?keyword=query`, so full search results currently focus on communities.

8. Moderator Flow

Moderators start at `/moderator/dashboard`.

Moderator dashboard shows:

- pending post count
- open report count
- ban count
- bulletin count

Moderators review content at `/moderator/approval-queue`.

Approval queue logic:

- load all pending posts
- moderator can approve or reject
- approve -> status = approved
- reject -> status = rejected
- optional rejection reason saved

- reviewed_by and reviewed_at are saved

Approved posts become visible in /member/home, /discover, community detail pages, and post detail pages. Rejected posts remain visible to the author in their own post list, but not public.

Other moderator screens:

- /moderator/reported-posts
- /moderator/banned-members
- /moderator/send-bulletin

Current limitations:

- Reports can be viewed, but user-facing report creation is not fully wired.
- Bulletins are stored and previewed, but not actually delivered as notifications.
- Voting logic exists as model/DAO placeholders, but voting is not fully implemented.

9. Admin Flow

Admins start at /admin/dashboard.

Admin dashboard shows:

- total users
- total communities
- pending posts
- open reports
- recent users
- recent posts

Admin management screens:

- /admin/manage-communities
- /admin/manage-users
- /admin/manage-moderators
- /admin/reports

Admin community flow:

- admin creates community
- system generates slug
- community requires review by default

Admin moderator flow:

- enter username
- choose community
- assign moderator
- user role becomes moderator

Removing a moderator assignment:

- remove assignment
- if user has no remaining moderator assignments, role becomes member

10. Main Data States

Post statuses:

- draft
- pending
- approved
- rejected
- removed

Community review model:

- requires_review = true
- posts must be approved before public display

User roles:

- member
- moderator
- admin

Ban logic:

- moderator can ban user from a community
- banned user cannot create posts in that community
- ban can be removed

11. Current End-To-End Loop

```
Visitor browses /discover
-> registers or logs in
-> member lands on /member/home
-> member browses all approved posts
-> member joins a community
-> member creates a post
-> post is saved as draft or submitted as pending
-> moderator reviews pending post
-> approved post appears in the home feed and public community pages
-> admin manages users, communities, moderators, and reports
```

12. Current Test Users

Role	Username	Email	Password
Admin	sociusadmin	admin@socius.app	Admin@1234

Moderator	ava_clarke	ava@socius.app	Ava@2026
Member	milan_green	milan@socius.app	Milan@2026